
【文档】README

■ YFramework

轻量级、面向服务的 Unity 游戏框架，基于依赖注入容器 GameContext 构建，提供开箱即用的核心系统，支持通过 partial 方法扩展业务逻辑。

| Unity 2022.3 · URP · C#

■ 项目结构

```
----- 代码 -----
YFramework/
|—— client/           # Unity 工程
|  |—— Assets/
|  |  |—— Scripts/
|  |  |—— Framework/  # 框架核心（与业务无关）
|  |  |—— Gameplay/   # 业务层（基于框架构建）
|  |  |—— Editor/     # 编辑器扩展
|  |  |—— ScriptGenerated/ # 配表自动生成代码
|  |—— Packages/
|—— excel/           # Excel 配置表源文件
|—— tools/           # 配表发布工具链（Python）
----- 代码 -----
```

■ 核心架构

◆ 启动流程

----- 代码 -----

```

GameLoop.Awake()
→ GameBootstrapper.BuildContext() // 注册所有服务
→ GameContext.InitAll()          // 按注册顺序初始化
→ RunProjectStartup()            // 业务启动入口
----- 代码 -----

```

◆ 帧循环

```

----- 代码 -----
GameLoop.Update()    → ITickable.Tick(dt)
GameLoop.FixedUpdate() → IFixedTickable.FixedTick(fdt)
GameLoop.LateUpdate() → ILateTickable.LateTick(dt)
----- 代码 -----

```

服务实现对应接口即可自动接收帧回调，关闭时按注册逆序执行 Shutdown()。

■ 框架模块

| 模块 | 类 | 说明 |

|-----|-----|-----|

| 服务容器 | GameContext | 类型安全的 DI 容器，Get<T>() / TryGet<T>() |

| 事件系统 | EventMgr | 零分配、类型安全的泛型事件，支持最多 4 参数 |

| 资源管理 | ResMgr | 引用计数 ResourceHandle<T>，同步/异步加载，场景切换自动卸载 |

| 对象池 | ObjectPool | 异步模板加载，时间衰减回收，逐帧释放限制 |

| UI 系统 | UIMgr | 5 层 Canvas 架构（Normal/Top/RayCast/Tips/PopText），页面生命周期管理 |

| 场景管理 | YSceneManager | 异步切换，YSceneBase 生命周期，支持 Loading 过渡 |

| 相机 | CameraMgr | Cinemachine 集成，射线点击/悬停/拖拽事件分发 |

| 音频 | SoundMgr | 多通道（BGM/SFX/UI），交叉淡入淡出，最多 16 路 SFX |

| 数据存储 | StoreMgr | DataContainer<T> 持久化，可替换存储驱动 |

| 寻路 | YAStarManager | A* Pathfinding 封装 |

| 状态机 | YStateMachine | 简易分层状态机，支持状态回退 |

| 定时器 | Timers | 全局定时器，支持无限循环 |

| 协程 | CoroutineRunner | 服务化协程管理 |

| 屏幕监听 | ScreenMonitor | 分辨率变化事件 |

■ 业务扩展方式

通过 GameBootstrapper 中的 partial 方法接入业务，无需修改框架代码：

```
----- 代码(csharp) -----  
// GameBootstrapper.Project.cs  
partial void RegisterProjectServices(GameContext ctx)  
{  
    ctx.Register(new PlayerManager());  
    ctx.Register(new EnemiesManager());  
}  
  
partial void ConfigureProjectScenes(YSceneManager sceneMgr)  
{  
    sceneMgr.RegisterScene<GameStartScene>();  
    sceneMgr.RegisterScene<GameMainScene>();  
}  
  
partial void ConfigureProjectUi(UIConfig uiConfig)  
{  
    uiConfig.Register<StartPanel>(UIEnum.StartPanel, UILayerEnum.Normal, "UI/StartPanel");  
}  
----- 代码 -----
```

■ UI 页面生命周期

```
----- 代码 -----  
OnLoad() → BeforeShow(param) → OnShow() → OnHide() → OnResize()  
----- 代码 -----
```

UIPageBase 提供 Context 和 UIManager 注入，通过 GetService<T>() 解析其他服务。

■ 配表工具链

Excel 配置表通过 Python 工具链发布为 Proto + C# 代码：

----- 代码 -----

Excel (excel/) → publish_config.py → .proto → protoc → C# 类 + 二进制数据

----- 代码 -----

配置文件: tools/tools_config.ini

----- 代码(bash) -----

发布配表

cd tools

python publish_config.py

----- 代码 -----

■ 主要依赖

- Cinemachine — 相机系统
- Universal RP — 渲染管线
- **A* Pathfinding** — 寻路
- TextMeshPro — UI 文本
- DOTween — 动画缓动
- Entities / Burst — 高性能计算
- SteamWorks.NET — Steam 集成

【文档】README

■ YFramework 规范文档

本目录收录三份规范，描述 YFramework 仓库的工程、代码、模块约定。三份文档相互独立，按以下顺序阅读最高效：

1. 项目规范（./项目规范.md） — 仓库结构、版本/依赖、资源约定、构建/发布、Git 协作。
2. 代码规范（./代码规范.md） — 命名、文件组织、注释、错误处理、性能、Unity 特定约定。
3. 模块规范（./模块规范.md） — 框架各模块的职责、对外 API、扩展约定、典型用法。

■ Skill workflow

规范是死的，落地靠 .claude/skills/ 下的 skill 链。当前两环：

| Skill | 触发 | 输入 | 输出 |

|---|---|---|---|

| code-planning | /code-planning <需求描述或策划案 id> | 自由文本需求 / 策划案/...md | 代码规划/
.../GP-Xxx-Plan-v1.md |

| code-generation | /code-generation <Plan id> | 代码规划/...md | client/Assets/Scripts/ 下的 .c
s 文件 |

约束要点：

- Framework 改动：两个 skill 都允许写 Framework/ 代码，但必须先经 code-planning 出规划且过其 §2.10 六条 Framework 设计原则（接口先行 / 向后兼容 / 职责单一 / 依赖方向单向 / 可池化可关闭 / 注册顺序）。直接修改 Framework 不走规划 = 违规。
- 依赖方向：仍然严格 Editor → Gameplay → Framework → Unity，详见项目规范 §1.1。
- 代码评审：见代码规范 §13；Framework 改动评审参考 code-planning skill §2.10。

■ 文档生成时的清理与重构

生成本规范的过程中，对一批无引用的死代码做了删除，并按规范执行了一轮结构整改：

死代码清理

| 路径 | 处理 | 理由 |

|---|---|---|

| Framework/UI/UITypeBase.cs | 删除 | 整文件被注释掉，无引用 |

| Framework/Pool/TestPoolObject.cs | 删除 | 测试样例，无任何引用 |

| Framework/GameContext/TestService.cs | 删除 | 空服务，仅在 GameBootstrapper 注册无逻辑 |

| Framework/Store/GameDataManager.cs | 删除 | 未注册到 GameContext，无引用 |

| StoreMgr.cs 内 ExampleUsage 区块 | 删除 | 残留示例 MonoBehaviour |

| GameBootstrapper.cs 中 TestService 注册 | 删除 | 配合上面 |

| Framework/Log/ 空目录 | 删除 | 占位无内容，规划中的 LogMgr 后续按需补回 |

结构整改

| 整改 | 内容 |

|---|---|

| SceneReferenceKeys 拆分 | 框架键 MainCamera/MainCameraVirtual/MainLight/AStarRoot 迁至 Framework/Scene/SceneRefKeys.cs; 业务键 PlayerSpawn/Spline 留在 Gameplay/Scene/GameSceneReferenceKeys.cs 并改类名 GameSceneRefKeys |

| EventMgr 与业务事件解耦 | YOTOEventType 枚举从 Framework/Event/EventConfig.cs 迁至 Gameplay/Event/GameEventTypes.cs; EventMgr 入参由具体枚举改为 System.Enum, 框架不再直接引用业务枚举 |

| 文件名 = 类名 | GotSceneBase.cs→YSceneBase.cs; GotSceneManager.cs→YSceneManager.cs; GotConfigManager.cs→ConfigManager.cs (连同 .meta 一并 git mv) |

| 目录拼写修正 | Framework/PathFinding/com.arongranberg.astar.enxtension/ → astar.extension/ |

| FlyText 服务上移 | 从 Gameplay/UI/FlyTexts/ 迁至 Framework/UI/FlyText/, 与其在 GameBootstrapper.BuildContext 中作为框架服务的注册位置一致 |

| 仓库根 README.md | 类名 GotSceneManager/GotAStarManager → YSceneManager/YAStarManager; partial 业务示例签名修正为实际签名 |

【文档】项目规范

■ 项目规范

| 范围: 仓库结构、Unity 版本/依赖、资源/场景规范、构建发布、配表流程、Git 协作。

■ 1. 仓库结构

----- 代码 -----

YFramework/

|—— ArtSource/ # 美术源文件 (PSD/AI/FBX/Blend 等), 不进 Unity 工程,

建议走 Git LFS

| |—— UI/ # 例: ChatFriend.psd / Inventory.psd 等 UI 原稿

|—— Docs/ # 本目录: 规范文档

|—— README.md # 项目总览

|—— 发布配表.bat # 一键发布配表

|—— client/ # Unity 工程根

```

| |—— Assets/
| | |—— Art/          # 美术：按"专业域"横向分类，全部资源都在这里
| | | |—— Animations/    # AnimController / .anim
| | | |—— Audio/
| | | | |—— BGM/        # 背景音乐
| | | | |—— SFX/        # 音效
| | | |—— Characters/    # 角色：模型/贴图/材质/动画 同业务子目录归并
| | | |—— Environments/  # 场景美术
| | | |—— Fonts/        # 字体（中文/英文/SDF）
| | | |—— Props/        # 道具
| | | |—— Shaders/      # .shader / .shadergraph
| | | |—— ThirdParty/    # 买来的整包美术资源，按"包名"目录保留作者结构（便于升级）
| | | |—— UI/          # 自制 UI 美术（图标/Atlas/Panels），第三方 UI 包放 Art
/ThirdParty 或 Art/UI/<包名>
| | | | |—— Atlas/      # SpriteAtlas
| | | | |—— Icons/      # 业务图标
| | | | |—— Panels/     # 自制 Panel 贴图
| | | | |—— HONETi/     # 第三方 HONETi UI 包整体保留（含 Canvas.prefab/Demo/Textures/<主题>）
| | | |—— VFX/         # 特效贴图/材质/Prefab
| | |—— Plugins/       # 第三方插件代码/SDK（DOTween/TextMeshPro/Astar/Steamworks/Protobuf）
| | |—— Resources/     # 运行时按路径加载的资源（受 Resources 约束）
| | | |—— Config/
| | | | |—— Astar/      # A* 寻路图（运行时加载）
| | | | |—— Data/      # 配表二进制 .bytes（自动生成）
| | | |—— UI/          # UI 预制体，按业务模块分子目录
| | | |—— Boot/        # LoadingPanel / StartPanel（启动相关）
| | | |—— Common/      # 通用组件（CommonButton / common_Item 等）
| | | |—— FlyText/     # FlyTextPrefab 等飘字
| | | |—— Main/        # GameMainPanel / FinishPanel（主玩法）
| | | |—— Setting/     # SettingPanel 及子面板
| | |—— ResourcesAssets/ # AB 包预留目录（未来通过 AssetBundle 加载的资源）
| | |—— Scenes/        # 场景文件，按生命周期阶段分类
| | | |—— Boot/        # 启动场景：GameStart.unity
| | | |—— Gameplay/    # 业务玩法场景
| | | |—— Persistent/  # 常驻场景：UISceneDontDelete.unity
| | | |—— Sandbox/     # 验证/调试用临时场景（不进 Build Settings）
| | |—— ScriptGenerated/ # 自动生成代码（人工不修改）

```

```

| | | |—— Config/          # Excel 解析类 (HeroConfig.cs 等)
| | | |—— Proto/           # protoc 生成的 *_pb2.cs
| | | |—— Scripts/         # 业务代码 (详见 §1.1)
| | | |—— StreamingAssets/
| | | |—— URP/             # 渲染管线设置
| |—— Packages/           # UPM 依赖
|—— ProjectSettings/      # Unity 工程设置
|—— client.sln            # 由 Unity 生成, 不手改
|—— excel/               # 配表源
|—— 3xlsx/              # *.xlsx 源文件
|—— tools/              # 配表发布工具链 (Python)
|—— publish_config.py    # 入口
|—— publish_single.bat   # 单表发布
|—— tools_config.ini     # 路径配置
|—— 配表工具复刻指南.md
|—— _publish/           # 中间产物
|—— 3rdparty/protobuf/protoc.exe # protoc 编译器
|—— gtp/                # 内部读写器
----- 代码 -----

```

◆ 1.0 角色—目录归属 (团队协作准则)

| 目录 | 主要负责角色 | 说明 |

|---|---|---|

| client/Assets/Art/ | 美术** | 美术全权管理; 程序只读引用, 不增改不重命名 |

| client/Assets/Resources/UI/** | 美术 + 程序 | 预制体由 UI 美术拼装; 程序按 §3.1 注册路径 |

| client/Assets/Resources/Config/** | 程序 (自动生成) | Data/*.bytes 由配表工具产出; 人工禁止改

|

| client/Assets/Scenes/** | 程序 + 关卡设计 | 业务场景搭建; 编辑期资源走 SceneReferenceProvider

|

| client/Assets/Scripts/ | 程序** | 严格分层 (§1.1) |

| client/Assets/ScriptGenerated/** | 工具链 | 配表/Proto 自动生成 |

| client/Assets/Plugins/** | 程序 | 第三方 SDK; 升级需走 PR |

| client/Assets/ResourcesAssets/** | 程序 | AB 包预留 |

| ArtSource/** (仓库根, Unity 工程外) | 美术 | PSD/AI/FBX/Blend 源文件; 建议 Git LFS |

◆ 1.1 Assets/Scripts/ 分层

----- 代码 -----

Scripts/

- |—— Framework/ # 框架核心，业务无关。**禁止反向依赖 Gameplay/**
- | |—— GameContext/ # DI 容器、GameLoop、IGameService、CoroutineRunner
- | |—— Camera/ # CameraMgr、SceneInteractionService
- | |—— Config/ # ConfigManager (partial 类，由 ScriptGenerated/Config 扩展)
- | |—— Event/ # EventMgr (接受任意 Enum 作为事件键，业务枚举见 Gameplay/Event)
- | |—— PathFinding/ # A* 封装 (YAStarManager 等，目录 com.arongranberg.astar.extension)
- | |—— Pool/ # ObjectPool / DataObjPool / ObjectBase / SceneModelBase
- | |—— Res/ # ResMgr (引用计数 + 异步加载)
- | |—— Scene/ # YSceneManager / YSceneBase / SceneReferenceService/Provider / Scene

RefKeys

- | |—— Sound/ # SoundMgr / SoundTypes
- | |—— StateMachine/ # YStateMachine
- | |—— Store/ # StoreMgr / DataContainer / 存储驱动
- | |—— Time/ # Timers
- | |—— UI/ # UIMgr / UIPageBase / UIPageHandler / UIConfig / Components / FlyTex

t

- | |—— DontDestroy.cs
- |—— Gameplay/ # 业务层，可依赖 Framework/，不可被 Framework/ 反向依赖
- | |—— GameProjectBootstrapper.cs # 业务接入点 (partial 实现)
- | |—— Event/ # 业务事件枚举 GameEventTypes
- | |—— Particle/ # ParticleEntity (基于 ObjectBase 的业务粒子)
- | |—— Scene/ # GameMainScene / GameStartScene / GameSceneRefKeys
- | |—— TaskSystem/ # TaskManager / TaskDefinition / TaskInstance
- | |—— UI/ # 各 *Panel / GameUIEnum
- |—— Editor/ # 编辑器扩展 (CustomEditor、批处理工具)
- |—— Steamworks.NET/ # 第三方 Steam SDK 接入

----- 代码 -----

依赖方向 (严格)：Editor → Gameplay → Framework → Unity / Packages。任何 Framework→Gameplay 的引用都视为问题。

Framework 改动权限：Framework 内部结构可被业务需求驱动扩展 (新增服务、给现有服务加重载、调整内部实现)，但必须先通过 .claude/skills/code-planning 出规划文档并满足其 §2.10 六条 Framework 设计原则 (接口先行 / 向后兼容 / 职责单一 / 依赖方向单向 / 可池化可关闭 / 注册顺序)。不在规划里的 Framework 改动一律不接受。

■ 2. Unity 版本与依赖

项	版本/说明
Unity	2022.3 LTS
渲染管线	URP
脚本运行时	.NET Standard 2.1 / IL2CPP (按平台)
目标帧率	60FPS (GameLoop.Awake 中 Application.targetFrameRate = 60)

核心包/插件 (详见 Packages/manifest.json) :

- Cinemachine — 虚拟相机管线, 由 CameraMgr 集成
- Universal RP — 渲染管线
- **A* Pathfinding Project** (com.arongranberg.astar) — 寻路; 扩展层在 Framework/PathFinding/com.arongranberg.astar.extension/
- TextMeshPro — UI 文本
- DOTween — 动画缓动 (按钮悬停/UI 渐显等)
- Entities / Burst / Mathematics — 高性能计算 (按需)
- Steamworks.NET — Steam 集成

升级 Unity 或核心插件版本必须:

1. 在 feature 分支单独提交;
2. PR 描述写清升级动机、影响面、测试覆盖;
3. 通过 GameStart.unity 完整跑一遍启动 → 进入 Gameplay 场景 → 退出, 确认无报错。

■ 3. 资源约定

◆ 3.1 路径与加载方式

资源类型	存放位置	加载入口	说明
UI 预制体 (启动/Loading)	Resources/UI/Boot/**	UIMgr.Show<TPage>()	注册路径示例 "UI/Boot/StartPanel"

UI 预制体（主玩法）	Resources/UI/Main/**	同上	路径示例 "UI/Main/GameMainPanel"
UI 预制体（设置）	Resources/UI/Setting/**	同上	路径示例 "UI/Setting/SettingPanel"
UI 预制体（通用组件）	Resources/UI/Common/**	同上 / ResMgr	公共 Item / Button
飘字	Resources/UI/FlyText/FlyTextPrefab	FlyTextCtrl	路径 "UI/FlyText/FlyTextPrefab"
配表数据	Resources/Config/Data/*.bytes	ConfigManager.heroConfig.Get(id) 等	自动生成，禁止人工放入
寻路图	Resources/Config/Astar/*	YAStarManager.LoadPathFinding	路径基址 Config/Astar/
音频	Resources/...（按业务路径）	SoundMgr.PlayBGM/PlaySFX	通过 ResMgr 引用计数管理
通用预制体	Resources/...	ResMgr.LoadHandleAsync / ObjectPool	走对象池或引用计数
美术原始资源	Art/、URP/	仅编辑期使用	不放进 Resources/ 避免打包膨胀
待 AB 包资源	ResourcesAssets/**	未来 AssetBundle.LoadFromFileAsync	当前不会被 Resources.Load 索引

◆ Map / 场景 prefab 资源摆放规则

Resources/Map/<MapName>/ 只放整体加载入口（即 XxxMap.prefab 这一个文件），它的材质、贴图、自制网格全部放 Art/Environments/, 靠 GUID 引用串起来：

资源	存放位置
Map 入口预制体	Resources/Map/<MapName>/<MapName>.prefab
Map 专用材质	Art/Environments/<MapName>/Materials/
Map 专用贴图	Art/Environments/<MapName>/Textures/
Map 自制网格	Art/Environments/<MapName>/Meshes/
跨 Map 共用材质/贴图	Art/Environments/_Shared/{Materials,Textures}/
通用 Shader	Art/Shaders/（不按 Map 分）

理由：Resources/ 下的资源会被 Unity 强制全量打入主包；贴图体积大，放进去会让包冗余膨胀。prefab → material → texture 是 GUID 引用，跨目录加载链不会断。

加载原则：

- 优先 LoadAsync / LoadHandleAsync，避免主线程同步 IO。
- 同一资源使用同一 path 字符串，确保 ResMgr 引用计数命中同一缓存项。
- 任何 Load/LoadHandle 调用必须有对应的 Release/Dispose，否则资源不会卸载。
- 切场景时，YSceneManager 会触发 ResMgr.OnChangeScene（含 UnloadUnusedAssets + GC），不需手工调用。

◆ 3.2 命名

- 文件夹：UpperCamelCase（Common/、Setting/）。
 - 预制体：业务前缀 + 用途，例如 StartPanel.prefab、FlyTextPrefab.prefab。
 - 脚本——对应 GameObject 时，名字保持一致。
-

■ 4. 场景规范

◆ 4.1 启动场景

- GameStart.unity 是入口场景，必须挂 GameLoop（DontDestroyOnLoad）。
- GameLoop.Awake 调用 GameBootstrapper.BuildContext → InitAll → RunStartup，业务应在 GameProjectBootstrapper.RunProjectStartup 中决定首屏（当前为 Show<StartPanel>()）。

◆ 4.2 业务场景

- 每个业务场景实现 YSceneBase，重写 SceneType、SceneName、OnEnterScene/OnLoadingEnd/OnLeaveScene。
- 在 GameProjectBootstrapper.ConfigureProjectScenes 中 RegisterScene<T>() 完成注册。
- 切换：ctx.Get<YSceneManager>().SwitchScene(YSceneType.GamePlay, args)，默认会显示 LoadingPanel 并在切完后 GC。

◆ 4.3 场景引用

场景里需要被代码访问的 GameObject 必须由 SceneReferenceProvider 暴露：

----- 代码(csharp) -----

// 不允许：

GameObject.Find("MainCamera");

// 推荐：在场景里挂 SceneReferenceProvider，配置 key→Transform 映射

ctx.Get<SceneReferenceService>().TryGetTransform("MainCamera", out var camTransform);

----- 代码 -----

GameObject.Find 仍在 SceneReferenceService 内做兜底，但仅作为最后手段，新代码请走 Provider。

■ 5. 配表发布流程

◆ 5.1 工具链

----- 代码 -----

excel/3xlsx/*.xlsx

└─ python tools/publish_config.py

└─→ tools/_publish/proto/*.proto (proto 定义)

└─→ client/Assets/ScriptGenerated/Config/*.cs (C# 解析类)

└─→ client/Assets/ScriptGenerated/Proto/*.cs (protoc 输出)

└─→ client/Assets/Resources/Config/Data/*.bytes (运行时数据)

----- 代码 -----

◆ 5.2 Excel 规则

- 表格放 excel/3xlsx/，文件名小写（hero.xlsx、quest.xlsx、skill.xlsx）。
- 每张表导出后产生 <Pascal>Config 类（hero.xlsx→HeroConfig），自动 partial 扩展到 ConfigManager。
- 跳过的文件配置在 tools/tools_config.ini 的 [publish] skip_files。
- 文件名以 ~ 或 \$ 开头（Excel 临时文件）会被自动跳过。

◆ 5.3 发布命令

----- 代码(bash) -----

全量发布

cd tools && python publish_config.py

或在仓库根

.\发布配表.bat

单表发布

```
. \tools\publish_single.bat hero
# 或
python tools/publish_config.py path/to/hero.xlsx
----- 代码 -----
```

◆ 5.4 工具链依赖

- Python 3.10+
- pip install protobuf grpcio-tools openpyxl
- protoc: 内置于 tools/3rdparty/protobuf/protoc.exe (Python 端用 grpc_tools.protoc, C# 端用此外置 protoc)

◆ 5.5 配表代码使用

ScriptGenerated/Config/ 内的代码 绝不手工修改。表头变更后必须重发布。

访问示例:

```
----- 代码(csharp) -----
var hero = ctx.Get<ConfigManager>().heroConfig.Get(1001);
Debug.Log(hero.HeroName);
----- 代码 -----
```

■ 6. 构建与打包

| 本节为约定，当前仓库尚未提交 CI 脚本。

- 启动场景必须是 GameStart.unity，并放在 Build Settings 第 0 位。
- 打包前必须先发布配表，确保 Resources/Config/Data/*.bytes 与表头一致。
- 平台特定开关（Steam、移动等）放在 ProjectSettings/，不通过编辑器手改后再回退。
- 不要 commit Library/、Temp/、obj/、Logs/ 等 Unity 生成目录（.gitignore 已覆盖）。

■ 7. Git 工作流

- 主分支：main（保护，仅 PR 合并）。
- 当前业务分支：T-Game（实际开发分支，按特性切子分支后回合）。
- 命名：feat/<feature>、fix/<bug>、refactor/<scope>、chore/<task>。
- 提交信息中文/英文均可，但保持动宾结构、单条 ≤72 字（参考 git log 现有风格）。
- 禁止 直接 push 到 main；禁止 --force 推送任何远程分支。
- .meta 文件必须随脚本/资源同步提交，不要遗漏，否则其他成员会看到 GUID 漂移。
- 大型二进制（音频/视频/贴图原图）走 LFS 或外部存储，不直接进 Git。

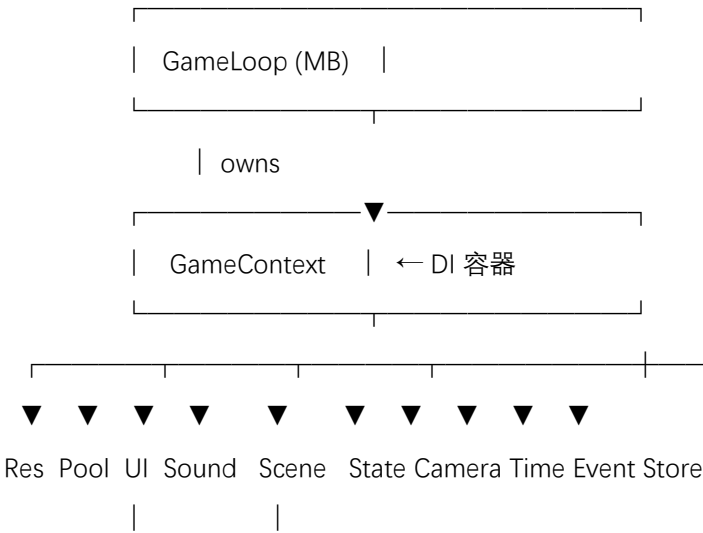
【文档】模块规范

■ 模块规范

- | 范围：Framework 各模块的职责、对外 API、扩展约定、典型用法。
- | 业务模块（GamePlay/）只在最后一节简述，业务侧实现细节随业务演进，不写死规范。

■ 0. 总览

----- 代码 -----



└─ UIPageHandler |

└─ YSceneBase 业务实现

----- 代码 -----

注册顺序定死在 GameBootstrapper.BuildContext (Framework 部分)，业务通过 partial 方法 RegisterProjectServices(ctx) 在末尾注入。

Init 按注册顺序执行，Shutdown 逆序执行。被依赖者在前。

■ 1. GameContext (DI 容器)

位置：Framework/GameContext/GameContext.cs

职责：服务注册、类型安全解析、生命周期 (Init / Shutdown)、帧分发 (Tick / FixedTick / LateTick)。

◆ 对外 API

----- 代码(csharp) -----

```
T Get<T>()                // 不存在抛异常
bool TryGet<T>(out T)      // 容错获取
void Register<T>(T instance) // 重复注册抛异常
void InitAll() / ShutdownAll() // 由 GameLoop 调用，业务勿直接用
void Tick(dt) / FixedTick(fdt) / LateTick(dt) // 由 GameLoop 调用
```

----- 代码 -----

◆ 扩展约定

- 新增框架服务：在 GameBootstrapper.BuildContext 适当位置 ctx.Register(new XxxMgr()) (依赖前置)。
- 新增业务服务：在 GameProjectBootstrapper.RegisterProjectServices(ctx) 添加。
- 服务必须实现 IGameService；如需帧调度则同时实现 ITickable 等。

■ 2. GameLoop / GameBootstrapper

位置：Framework/GameContext/GameLoop.cs、GameBootstrapper.cs、GamePlay/GameProjectBootstrapper.cs

职责：

- GameLoop：MonoBehaviour 入口，Awake 时构建 Context、InitAll、RunStartup；Update/FixedUpdate/LateUpdate 转发到 Context；OnDestroy 调 ShutdownAll。
- GameBootstrapper：partial static 类。Framework 写死框架服务注册；业务通过四个 partial 方法插入：

----- 代码(csharp) -----

```
static partial void RegisterProjectServices(GameContext ctx);  
static partial void ConfigureProjectScenes(YSceneManager sceneManager);  
static partial void ConfigureProjectUi(UIConfig uiConfig);  
static partial void RunProjectStartup(GameContext ctx);
```

----- 代码 -----

约定：

- RunProjectStartup 是业务首屏入口（当前为 Show<StartPanel>()），不要在 RegisterProjectServices 里调 UI / 触发资源加载。
- 不要在业务侧重新定义 BuildContext；通过 partial 注入即可。

■ 3. CoroutineRunner

位置：Framework/GameContext/CoroutineRunner.cs

接口：ICoroutineRunner { Coroutine Run(IEnumerator); void Stop(Coroutine); }

注册形式：MonoBehaviour，挂在 GameLoop 同一 GameObject 上（BuildContext 自动 AddComponent）。

用法：所有需要协程的服务在 Init 缓存 ICoroutineRunner，Shutdown 置空。禁止自己挂 MonoBehaviour 起协程。

■ 4. 事件系统 EventMgr

位置：Framework/Event/EventMgr.cs（业务枚举见 Gameplay/Event/GameEventTypes.cs）

职责：类型安全的全局事件，泛型最多 4 参数；同回调防重复订阅。事件键为任意 System.Enum，框架不绑定具体业务枚举。

◆ API

----- 代码(csharp) -----

// 订阅 / 反订阅（0-4 参数泛型重载）

void Add<T1,T2>(Enum type, Action<T1,T2> callback)

void Remove<T1,T2>(Enum type, Action<T1,T2> callback)

// 触发

void Trigger<T1,T2>(Enum type, T1 a, T2 b)

void Clear()

----- 代码 -----

调用时直接传枚举值（如 YOTOEventType.RefreshBagList），编译期推导类型；Enum 仅作内部 dictionary 键。

◆ 实现关键

- 内部用 Dictionary<Enum, IEventSlot> + EventSlot<TDelegate> 持有委托。
- 订阅时类型不匹配会 LogError（同一事件键一旦确定参数签名，后续订阅必须一致）。
- 反订阅后若槽位为空自动移除，避免内存膨胀。

◆ 约定

- 生命周期：UI 页面 OnShow 订阅、OnHide 反订阅；服务 Init / Shutdown。
- 不要用匿名 lambda 直接订阅（无法准确反订阅）；用具名方法或 Action 字段。
- 多于 4 参数 → 定义结构体作为单参传入。
- 业务枚举集中在 Gameplay/Event/GameEventTypes.cs（当前文件中的 YOTOEventType），新业务事件加在那里；不要在框架 Framework/Event/ 下定义业务事件。

■ 5. 资源管理 ResMgr

位置：Framework/Res/ResMgr.cs

职责：基于 Resources.Load 的引用计数 + 异步加载 + ResourceHandle<T> 自动释放。

◆ API

----- 代码(csharp) -----

```
T Load<T>(string path)           // 同步, 计数 +1
void LoadAsync<T>(string path, Action<T> cb) // 异步, 计数 +1
ResourceHandle<T> LoadHandle<T>(string path) // 同步, 返回 handle (推荐)
void LoadHandleAsync<T>(string path, Action<ResourceHandle<T>> cb) // 异步 + handle
void Release<T>(string path)       // 计数 -1
void ReleasePack(string path, Object obj = null) // 按对象类型自动 Release
IEnumerator OnChangeScene(Action cb) // 切场景时清理 (YSceneManager 自动)
----- 代码 -----
```

◆ ResourceHandle<T>

- IDisposable, using 块内自动 Release。
- 持有 Path、Asset、IsReleased。
- 子系统 (UIPageHandler、SoundMgr、ObjectPool.PoolBuffer) 全部用 handle, 确保资源生命周期可追踪。

◆ 关键不变量

- 资源以 (path, Type) 作为缓存 key。同 path 但不同类型加载产生独立条目。
- 异步加载未完成期间多次请求合并为同一 coroutine + pending callbacks, 不会重复发起 LoadAsync。
- Resources.UnloadAsset 不能用于 GameObject/Component, 所以 GameObject 释放时只是从缓存移除并标记 pendingUnusedAssetSweep, 由下次 OnChangeScene 调用 Resources.UnloadUnusedAssets 完成。

◆ 约定

- 始终通过 ResMgr，不直接 Resources.Load。
- 加载失败：API 返回 null，调用方判空。
- 不缓存 ResourceHandle.Asset 字段超过 handle 生命周期。

■ 6. UI 系统

位置：Framework/UI/UIMgr.cs、UIConfig.cs、UIPageBase.cs、UIPageHandler.cs、Components/

◆ 6.1 分层

5 层 Canvas (UILayerEnum)，渲染顺序由低到高：

层	sortingOrder	用途
Normal	0	主界面、面板 (StartPanel、GameMainPanel)
Top	100	普通弹窗
RayCast	200	阻挡输入的全屏遮罩 (LoadingPanel 当前在此层)
Tips	300	提示弹窗
PopText	400	飘字 (FlyText)

每层独立 Canvas + CanvasScaler (1920×1080 参考分辨率, MatchWidthOrHeight=0) + GraphicRaycaster。

◆ 6.2 注册流程

业务在 GameProjectBootstrapper.ConfigureProjectUi 中注册：

```
----- 代码(csharp) -----  
uiConfig.RegisterLoading<LoadingPanel>(UIEnum.LoadingPanel, UILayerEnum.RayCast, "UI/Loading  
Panel", 0f);  
uiConfig.Register<StartPanel>(UIEnum.StartPanel, UILayerEnum.Normal, "UI/StartPanel");  
----- 代码 -----
```

参数：枚举 / 层级 / Resources 路径 / 关闭后销毁延迟（秒）。closeDestroyDelay <= 0 表示不自动销毁

。

◆ 6.3 显示 / 隐藏

----- 代码(csharp) -----

// 通过容器

```
var ui = ctx.Get<UIMgr>();
```

```
ui.Show<StartPanel>();
```

```
ui.Show<MyPanel>(myParam);
```

```
ui.Hide<MyPanel>();
```

```
ui.ShowLoading();
```

```
ui.HideLoading();
```

```
ui.IsShown(UIEnum.StartPanel);
```

```
ui.TryGetPage<StartPanel>(out var page);
```

// 在 UIPageBase 内

```
Show<MyPanel>();
```

```
CloseSelf();
```

----- 代码 -----

◆ 6.4 页面生命周期

----- 代码 -----

预制体 prefab 异步加载

↓

```
Initialize(context, manager) // 注入依赖
```

↓

```
OnLoad() // 一次性绑定
```

↓

```
BeforeShow(param) / OnBeforeShow<TParam>
```

↓

```
OnShow() // 订阅事件、刷新数据
```

↓

...展示中...

↓

```
OnHide() // 反订阅、停 Tween
```

↓

```
(closeDestroyDelay 后)
Destroy()           // 销毁 GameObject、Release prefab handle
----- 代码 -----
```

OnResize 在屏幕分辨率变化时（ScreenMonitor.OnScreenResize → UIMgr.ResizeScreen）逐页调用。

◆ 6.5 UIPageBase

- Awake 自动收集所有 YOTOUIChangeBase Tween（YOTOUIShow 等），Enter/Exit 时统一播放。
- GetComponent(typeof(CanvasGroup), typeof(YOTOUIShow))：预制体 必须 挂这两个组件。
- 泛型版 UIPageBase<TParam>：自动把 BeforeShow(object) 转换为 OnBeforeShow(TParam)，类型不匹配会 LogError。

◆ 6.6 复用组件（Components/）

| 组件 | 用途 |

|---|---|

| YOTOButton | 含悬停/点击缩放 + 文字颜色三态的按钮，配 YOTOButtonEditor |

| YOTOScrollView | 池化滚动列表（垂直/水平/Grid） |

| YOTOScrollViewItem | 列表 item 基类 |

| YOTOUIShow | CanvasGroup 渐显/渐隐（OnEnter/OnExist） |

| YOTOUIMove | 位置移动型动画 |

| YOTOUIChangeBase | 上述动画的抽象基类 |

◆ 6.7 屏幕监听

ScreenMonitor : ITickable 每帧比较 Screen.width/height，变化时触发 OnScreenResize 事件，业务可订阅或交给 UIMgr.ResizeScreen 自动处理。

◆ 6.8 飘字 FlyText

位置：Framework/UI/FlyText/（FlyTextMgr.cs、FlyTextCtrl.cs）。

- FlyTextMgr : IGameService, ITickable —— 在 BuildContext 中作为框架服务注册，每帧最多生成 generateNumPerFrame 个飘字，超量入队等待。
- FlyTextCtrl : ObjectBase, PoolItem<Transform> —— 单条飘字实体，复用 DataObjPool，飘字结束自动

回池。

- 业务调用:

```
----- 代码(csharp) -----  
var fly = ctx.Get<FlyTextMgr>();  
fly.AddText("hit!", worldPos);           // 默认 Normal, 世界坐标  
fly.AddTextAtScreenCenter("CRITICAL", FlyTextType.Quick);  
----- 代码 -----
```

- 飘字预制体路径硬编码 UI/FlyText/FlyTextPrefab (在 FlyTextCtrl.SetData) , 实际位于 Resources/UI/FlyText/FlyTextPrefab.prefab。
 - FlyTextType 决定颜色与动画风格 (Normal/Quick/PlayerHurt/AddHP) 。
-

■ 7. 场景管理

位置: Framework/Scene/、业务侧 Gameplay/Scene/

◆ 7.1 YSceneManager

职责: 业务场景注册、切换、Loading 过渡、切场景 GC。

API:

```
----- 代码(csharp) -----  
void RegisterScene<T>() where T : YSceneBase, new()  
void SwitchScene(YSceneType type, object args = null,  
                bool showLoading = true,  
                LoadSceneMode mode = LoadSceneMode.Single)  
void ForceUnloadCurrentScene()  
YSceneBase CurrentScene { get; }  
YSceneType PreSceneType { get; }  
bool SwitchSceneComplete { get; }  
----- 代码 -----
```

SceneRoot 是一个根 GameObject, 所有场景实例化为它的子节点 (SetActive(false) 隐藏) 。

◆ 7.2 YSceneBase

继承时实现：

----- 代码(csharp) -----

```
public override YSceneType SceneType => YSceneType.GamePlay;
```

```
public override string SceneName => "GameMainScene";
```

```
protected override void OnEnterScene() { /* 必须最终调 EnterSceneComplete() */ }
```

```
protected override void OnLoadingEnd() { /* 资源到位、UI 显示 */ }
```

```
protected override void OnLeaveScene() { /* 必须最终调 LeaveSceneComplete() */ }
```

----- 代码 -----

切换流程：

----- 代码 -----

SwitchScene

→ 上一场景 LeaveScene (异步) → resMgr.OnChangeScene (UnloadUnusedAssets + GC)

→ CurrentScene.EnterScene → OnEnterScene → EnterSceneComplete

→ SceneReferenceService.InvalidateCache

→ UIMgr.InjectSceneModels

→ CurrentScene.LoadingEnd → OnLoadingEnd

→ UIMgr.HideLoading

→ SwitchSceneComplete = true

----- 代码 -----

◆ 7.3 SceneReferenceProvider / Service

思路：场景里需要被代码访问的 Transform / Light，挂 SceneReferenceProvider (key→Transform 列表)，通过 SceneReferenceService.TryGetTransform(key, ...) 取。失败兜底 GameObject.Find。

使用规则：

- 框架级 key 常量定义在 Framework/Scene/SceneRefKeys.cs 的 SceneRefKeys 类 (MainCamera、MainCameraVirtual、MainLight、AStarRoot)。
- 业务级 key 定义在 Gameplay/Scene/GameSceneReferenceKeys.cs 的 GameSceneRefKeys 类 (PlayerSpawn、Spline)，按需新增。

- 新代码不要用 GameObject.Find, 用 Provider。

■ 8. 对象池

位置: Framework/Pool/

◆ 8.1 ObjectPool (GameObject 池)

职责: 异步加载 prefab 模板 (ResMgr.LoadHandleAsync), 以 prefab 路径为 key 维护多个 PoolBuffer, 每个 buffer 独立 loopCheckCDTime、inactiveTimeMax、perFrameDisposeCountMax、poolSizeMax。

API (间接, 由 ObjectBase 子类调用):

----- 代码(csharp) -----

```
ObjectPool.PoolBuffer GetBuffer(name, loopCD, inactiveMax, perFrameDispose, poolSizeMax)
buffer.AsyncLoadAndGetItem(callback)
buffer.RecoverItem(gameObject)
```

----- 代码 -----

Init 时自动 Timers.inst.Add(0.5f, -1, LoopCheck), 每 0.5 秒回收过期对象。

◆ 8.2 ObjectBase (场景对象基类)

继承形式:

----- 代码(csharp) -----

```
public class MyEntity : ObjectBase, PoolItem<MyData> { ... }
```

----- 代码 -----

实现:

- AfterIntoObjectPool() — 入池前重置属性。
- SetData(MyData) — 取出时设置数据; 典型流程: SetPrefabBundlePath(path) + InstanceGObj()。
- AfterInstanceGObj() — prefab 实例化完成回调 (应用初始 transform、播动画)。
- BeforeRecover(bool isDelete) — 回收前清理 (停定时器、取消事件)。

◆ 8.3 DataObjPool<T,S> (数据对象池)

纯 C# 对象 (无 GameObject) 的池:

```
----- 代码(csharp) -----  
public static DataObjPool<MyEntity, MyData> pool =  
    new DataObjPool<MyEntity, MyData>("MyEntity", capacity: 4);  
  
var item = pool.GetItem(myData); // 命中率统计 HitRate  
pool.RecoverItem(item);  
----- 代码 -----
```

◆ 8.4 SceneModelBase

ObjectBase 的 GameObject 实例化时, 自动挂载 SceneModelBase (已有则复用), 把以下接口转发到 ObjectBase:

- 物理: I2D/3DColliderHandler、I2D/3DTriggerHandler
- 输入: IClickable、IHoverable、IDraggable (由 SceneInteractionService 驱动)

UIMgr.InjectSceneModels 在切场景后注入 IUIService, 业务对象因此可在场景里访问 UI (少量场景需要)。

◆ 8.5 BaseEntity

简单的 ID 生成 + Init/Hide 钩子的轻量基类, 配合 DataObjPool (在 GetItem/RecoverItem 中检测)。

■ 9. 音频 SoundMgr

位置: Framework/Sound/

通道: SoundChannel { Master, Music, Sfx, Ui }, 4 通道独立音量与静音。

- Music: 1 路 AudioSource, 交叉淡入淡出 (默认 0.15s)。
- SFX: 1–16 路 (按需扩到 MaxSfxEmitters=16)。
- UI: 复用 SFX 池, 但通道独立计算音量。

API:

```

----- 代码(csharp) -----
PlayBGM(path, volume)
StopBGM(fadeDuration = 0.1f)
PlaySFX(path, volume)
PlayUISFX(path, volume)
Play(path, SoundPlayOptions)    // 通用入口

PlayMusic(path, volumeScale, fadeDuration, restartIfSame, loop)
StopMusic(fade)
StopAllSFX(); StopAll();

GetChannelVolume(ch); IsChannelMuted(ch);
SetMasterVolume / SetBgmVolume / SetSfxVolume / SetUiVolume
SetMasterMuted / SetMusicMuted / SetSfxMuted / SetUiMuted
SetChannelVolume(ch, v); SetChannelMuted(ch, m);
----- 代码 -----

```

设置变化自动通过 SoundSettingsContainer : DataContainer<SoundSettingsData> 持久化 (key=sound_settings)。

约定:

- 资源路径必须在 Resources/ 下, SoundMgr 内部用 ResMgr.LoadHandleAsync<AudioClip>, 自动计数。
- Stop* 后 AudioClip handle 自动 release。
- BGM 在不同的 path 间切换会 StopMusicTransition + SwapMusic 协程, 确保不会因网络/异步导致同时多个 music 在播。

■ 10. 数据存储 StoreMgr

位置: Framework/Store/StoreMgr.cs

◆ 架构

----- 代码 -----

DataContainer<T> ← 业务定义存档结构

|

| Save / Load

▼

StoreMgr → ISaveStrategy (默认 JsonSaveStrategy / Unity JsonUtility)

IStrorageDriver (默认 FileStorageDriver / persistentDataPath)

|

└─ ICoroutineRunner (异步 IO)

----- 代码 -----

◆ 业务用法

----- 代码(csharp) -----

[Serializable]

```
public class MyData { public int level; }
```

```
public class MyContainer : DataContainer<MyData>
```

```
{
```

```
    private MyData data = new();
```

```
    public override string SaveKey => "my_save";
```

```
    public override MyData GetData() => data;
```

```
    public override void __SetData(MyData v) => data = v ?? new MyData();
```

```
}
```

// 在某个服务中:

```
var c = new MyContainer();
```

```
c.BindStore(ctx.Get<StoreMgr>());
```

```
c.Load(() => Debug.Log(c.GetData().level));
```

```
c.Save();
```

----- 代码 -----

◆ 扩展

- 替换序列化器：实现 ISaveStrategy（如 MessagePack、Protobuf），改 StoreMgr.Init 内的策略实例。
 - 替换存储介质：实现 IStorageDriver（如 PlayerPrefs、SQLite、云存储），同上。
 - 默认存储路径：Application.persistentDataPath/<saveKey>.json。
-

■ 11. 相机 CameraMgr / SceneInteractionService

位置：Framework/Camera/

◆ CameraMgr

- 通过 SceneReferenceService 解析 MainCamera 的 Transform。
- 缓存 Cinemachine VirtualCamera / FreeLook，按 name 取。
- 提供震屏 ShakeCamera(duration, intensity) + UpdateShake(dt)（由 SceneInteractionService.Tick 间接驱动）。
- UseVirtualCamera = true 时，Init 内自动给主相机加 CinemachineBrain、生成 MainCameraVirtual。

◆ SceneInteractionService

ITickable，每帧只做一次 Physics.Raycast 缓存命中物，分发：

- 悬停 → IHoverable.OnHover/OnHoverExit
- 点击 → IClickable.OnClick（不超过拖拽阈值时）
- 拖拽 → IDraggable.OnMouseDown/Drag/Up（用平面投射避免出碰撞体中断）

UI 上方点击会 EventSystem.IsPointerOverGameObject 直接屏蔽，不进入场景交互。

排除层：自动忽略 BulletTrigger 层（如不存在则用 DefaultRaycastLayers）。

■ 12. 寻路 YAStarManager / IYPathFindingManager

位置：Framework/PathFinding/com.arongranberg.astar.extension/

◆ 接口

```
----- 代码(csharp) -----  
interface IYPathFindingManager {  
    void LoadPathFinding(UnityAction onComplete, string[] graphList);  
    void UnloadPathFinding();  
    void SetGraph(string path, UnityAction cb);  
    void AddGraph(string path, UnityAction cb);  
    void ClearGraph();  
}  
----- 代码 -----
```

◆ 抽象层（与 A* 解耦）

- IYSeeker — AI 移动入口（OncePathFinding、TP、AddForce、StopPathFinding 等）。
- IYObstacle — 动态障碍。
- IYLinker — 跳跃链接。
- PathFindingSeekerConfig 派生：AStarHighSeekerConfig / Mid / Low，分别对应不同精度（重力/避障/平滑）。

◆ 用法

业务通过 `ctx.Get<YAStarManager>()` 拿到管理器；具体 seeker / obstacle 创建走对应 Got* 实现类（GotAStarSeeker、GotAStarObstacle、GotAStarLinker）。

寻路图基址 Resources/Config/Astar/，运行时通过 ResMgr 加载。

■ 13. 状态机 YStateMachine

位置：Framework/StateMachine/YStateMachine.cs

◆ 5.2 ExcelReader.py — 读取 Excel

```
----- 代码(python) -----  
# gtp/reader/ExcelReader.py  
  
import openpyxl  
import os  
from gtp.common.ConfigData import (  
    ConfigData, isValidType, getPythonValue, ARRAY_SEP  
)  
  
def _getStdType(typeStr):  
    """标准化类型字符串"""  
    typeStr = (typeStr.lower().strip()  
        .replace("_", ".")  
        .replace("dictionary", "map")  
        .replace("vector2", "vec2")  
        .replace("vector3", "vec3")  
        .replace("vec3.array", "array.vec3")  
        .replace("vec2.array", "array.vec2"))  
    if isValidType(typeStr):  
        return typeStr  
    raise Exception("无效的数据类型: " + typeStr)  
  
def _getStdTarget(targetStr):  
    """将 Row4 目标文本转为缩写"""  
    targetStr = targetStr.lower().strip()  
    if "server" in targetStr:  
        return "S"  
    elif "client" in targetStr:  
        return "C"  
    elif "all" in targetStr:  
        return "CS"  
    return ""  
  
def readExcel(path):
```

.....

读取一个 Excel 配置表，返回 ConfigData 对象。

行结构：

Row 1 = 列显示名, Row 2 = 列ID, Row 3 = 类型,

Row 4 = 目标/键, Row 5 = 扩展(忽略),

Row 6 = 默认值, Row 7+ = 数据

树形表：Row 7 = 默认值, Row 8+ = 数据

.....

```
wb = openpyxl.load_workbook(filename=path, read_only=True, data_only=True)
```

```
st = wb.worksheets[0]
```

```
r = ConfigData()
```

```
r.id = os.path.splitext(os.path.basename(path))[0]
```

```
r.name = r.id
```

```
# ---- 1. 列检测 (Row 2) ----
```

```
cells = st[2]
```

```
detectionCount = 10
```

```
noneCount = 0
```

```
cellIndex = 1
```

```
mapTables = {} # {colSeq(str): actual_excel_column(int)}
```

```
for tmpcell in cells:
```

```
    if tmpcell.data_type == 's':
```

```
        noneCount = 0
```

```
        cellVal = tmpcell.value.strip()
```

```
        if cellVal != "":
```

```
            r.colID.append(cellVal)
```

```
            mapTables[str(len(r.colID))] = cellIndex
```

```
        else:
```

```
            break
```

```
    else:
```

```
        noneCount += 1
```

```
        if noneCount == detectionCount:
```

```
            break
```

```
    cellIndex += 1
```

```
colCount = len(r.colID)
```

```
if colCount == 0:
```

```
    wb.close()
```

```
    return None
```



```

# ---- 2. 读取列元数据 ----
indexC = []
isTree = False

for colIndex in range(1, colCount + 1):
    curCol = mapTables[str(colIndex)]

    # Row 1: 列名
    r.colName.append(str(st.cell(row=1, column=curCol).value or "").strip())

    # Row 3: 类型
    typeStr = str(st.cell(row=3, column=curCol).value or "").strip()
    r.colType.append(_getStdType(typeStr))

    # Row 4: 目标+键
    row4Str = str(st.cell(row=4, column=curCol).value or "").strip()

    extraFlag = ""
    if "main" in row4Str:
        extraFlag += "M"

    target = _getStdTarget(row4Str)
    r.colTarget.append(target + extraFlag)

    if "rowkey" in row4Str:
        r.ext.append("R")

    # 索引收集: 仅客户端
    if "key" in row4Str:
        if "all" in row4Str or "client" in row4Str:
            indexC.append(r.colID[colIndex - 1])

    # 树形检测
    if "main" in row4Str or "child" in row4Str or "row" in row4Str:
        isTree = True

if len(indexC) > 0:
    r.dataIndexC.append(indexC)

```

```

# ---- 3. 读取默认值 ----
defaultRow = 6
for colIndex in range(1, colCount + 1):
    curCol = mapTables[str(colIndex)]
    cell = st.cell(row=defaultRow, column=curCol)

    # 树形表: Row 7 可覆盖 Row 6
    if isTree:
        cell2 = st.cell(row=7, column=curCol)
        if cell2.value is not None:
            cell = cell2

    colType = r.colType[colIndex - 1]
    cellVal = cell.value

    # 分隔符替换
    if cellVal is not None and isinstance(cellVal, str):
        if "map" in colType:
            cellVal = cellVal.replace(";", ARRAY_SEP).replace("=", "~")
        elif "vec" in colType:
            cellVal = cellVal.replace(";", "~")

    r.colDefault.append(getPythonValue(cellVal, colType, None))

# ---- 4. 读取数据行 ----
startRow = 7 if not isTree else 8
rowIndex = 0
lastRowData = None

for row in st.rows:
    rowIndex += 1
    if len(row) == 0:
        break
    if rowIndex < startRow:
        continue

    rowData = []
    nullCount = 0

    for colIndex in range(1, colCount + 1):

```

```

curCol = mapTables[str(colIndex)]
cell = row[curCol - 1]
cellVal = cell.value

if cellVal is None:
    nullCount += 1

colType = r.colType[colIndex - 1]

# 分隔符替换
if cellVal is not None and isinstance(cellVal, str):
    if "map" in colType:
        cellVal = cellVal.replace(";", ARRAY_SEP).replace("=", "~")
    elif "vec" in colType:
        cellVal = cellVal.replace(";", "~")

# 树形继承
if isTree and "M" in r.colTarget[colIndex - 1] and lastRowData is not None:
    rowData.append(getPythonValue(cellVal, colType, lastRowData[colIndex - 1]))
else:
    rowData.append(getPythonValue(cellVal, colType, r.colDefault[colIndex - 1]))

if nullCount >= colCount:
    break

r.data.append(rowData)
lastRowData = rowData

wb.close()
return r
----- 代码 -----

```

◆ 5.3 ProtoHelper.py — 类型映射

```

----- 代码(python) -----
# gtp/writer/ProtoHelper.py

# 配置类型 → Proto 类型
type_map = {

```

```

"uint":      "uint32",
"int":       "int32",
"bool":      "int32",
"string":    "string",
"float":     "float",
"vec2.int":  "Vector2Int",
"vec2.float": "Vector2Float",
"vec3.int":  "Vector3Int",
"vec3.float": "Vector3Float",
"array.uint": "uint32",
"array.int":  "int32",
"array.float": "float",
"array.string": "string",
"array.bool": "int32",
"array.vec2.int": "Vector2Int",
"array.vec2.float": "Vector2Float",
"array.vec3.int": "Vector3Int",
"array.vec3.float": "Vector3Float",
"map.string.int": "KeyValuePairStrInt",
"map.string.float": "KeyValuePairStrFloat",
}

```

```

def getProtoType(typeStr):
    """返回 proto 字段类型"""
    return type_map.get(typeStr, typeStr)

```

```

def getProtoRule(typeStr):
    """返回 proto 字段规则: array/map 用 repeated, 其余 optional"""
    if "array" in typeStr or "map" in typeStr:
        return "repeated"
    return "optional"
----- 代码 -----

```

◆ 5.4 ProtoWriter.py — 生成 .proto 文件

```

----- 代码(python) -----
# gtp/writer/ProtoWriter.py

```

```

from gtp.writer import ProtoHelper

# ---- 基础类型定义（只需生成一次） ----
PROTO_BASE_TYPES = '''/**
 * Generated By Tool, DO NOT MODIFY.
 **/
syntax = "proto2";
package com.youzu.got.config;

message Vector2Int {
    optional int32 x = 1;
    optional int32 y = 2;
}

message Vector3Int {
    optional int32 x = 1;
    optional int32 y = 2;
    optional int32 z = 3;
}

message Vector2Float {
    optional float x = 1;
    optional float y = 2;
}

message Vector3Float {
    optional float x = 1;
    optional float y = 2;
    optional float z = 3;
}

message KeyValuePairStrInt {
    optional string k = 1;
    optional int32 v = 2;
}

message KeyValuePairStrFloat {
    optional string k = 1;
    optional float v = 2;
}

// ---- Header 索引类型（用于新版懒加载） ----
message IndexInt {
    optional int32 id = 1;

```

```

    optional int32 offset = 2;
}
message ConfigIntHeader {
    repeated IndexInt headers = 1;
}
message IndexUint {
    optional uint32 id = 1;
    optional int32 offset = 2;
}
message ConfigUintHeader {
    repeated IndexUint headers = 1;
}
message IndexStr {
    optional string id = 1;
    optional int32 offset = 2;
}
message ConfigStringHeader {
    repeated IndexStr headers = 1;
}
'''

```

```

def writeBaseTypes(path):
    """生成 ConfigBaseType.proto"""
    with open(path, 'w', encoding='utf-8') as f:
        f.write(PROTO_BASE_TYPES)

```

```

def writeData(path, data):
    """为一张配置表生成 .proto 文件"""
    colCount = len(data.colID)
    if colCount <= 0:
        return False

    className = data.id[0].upper() + data.id[1:]
    package = "com.youzu.got.config"

    lines = []
    lines.append('/**\n * Generated by tools, don\'t modify!\n**/')
    lines.append('syntax = "proto2";')

```

```

lines.append('package %s;\n' % package)
lines.append('import "ConfigBaseType.proto";\n')
lines.append('message %s {' % className)

for i in range(colCount):
    rule = ProtoHelper.getProtoRule(data.colType[i])
    ptype = ProtoHelper.getProtoType(data.colType[i])
    lines.append('  %s %s %s = %d; // %s' % (
        rule, ptype, data.colID[i], i + 1, data.colName[i]
    ))

lines.append('}\n')

# LIST 包装消息（用于序列化整张表）
lines.append('message %s_LIST_TOOL_RESERVED {' % className)
lines.append('  repeated %s items = 1;' % className)
lines.append('}')

with open(path, 'w', encoding='utf-8') as f:
    f.write('\n'.join(lines))
return True

```

----- 代码 -----

◆ 5.5 ProtoDataWriter.py — 生成 .bytes 二进制数据

----- 代码(python) -----

```

# gtp/writer/ProtoDataWriter.py

import struct
import importlib
from io import BytesIO
from gtp.writer import ProtoHelper

def _getProtoValue(fieldType, cellVal, isRepeated=False):
    """将 Python 值转为 proto 可赋值的对象"""
    # 需要先用 protoc 编译 .proto 生成 _pb2.py, 然后 import 使用
    # 这里给出逻辑伪代码, 实际使用需要 import 对应的 pb2 模块
    pass

```

```
def writeData(path, data, pb2_module):
```

```
    """
```

将 ConfigData 序列化为 .bytes 二进制文件。

文件格式 (大端序):

2 bytes: mainKeyCount (uint16 BE)	
2 bytes: headerCount (uint16 BE)	
4 bytes: headerLength (uint32 BE)	
[headerLength bytes: ConfigHeader]	
[Entry1: 2B长度 + protobuf二进制]	
[Entry2: 2B长度 + protobuf二进制]	
...	

每条 Entry 的偏移量记录在 Header 中,

C# 读取时 Seek 位置 = headerLength + 8 + header.offset

```
    """
```

```
    className = data.id[0].upper() + data.id[1:]
```

```
    colCount = len(data.colID)
```

```
    rowCount = len(data.data)
```

```
    if colCount <= 0 or rowCount <= 0:
```

```
        return False
```

```
    # 获取 pb2 模块中的消息类
```

```
    ItemClass = getattr(pb2_module, className)
```

```
    ListClass = getattr(pb2_module, className + "_LIST_TOOL_RESERVED")
```

```
    # 确定主键列索引
```

```
    mainKeyCol = data.colID.index(data.dataIndex[0][0]) if data.dataIndex else 0
```

```
    subKeyCol = None
```

```
    if data.dataIndex and len(data.dataIndex[0]) > 1:
```

```
        subKeyCol = data.colID.index(data.dataIndex[0][1])
```

```
    # ---- Stage 1: 序列化每条数据 ----
```

```
    entries_buffer = BytesIO()
```

```
    offsets = []
```



```

for rowIdx in range(rowCount):
    row = data.data[rowIdx]
    item = ItemClass()

    for colIdx in range(colCount):
        cellVal = row[colIdx]
        if cellVal is None:
            continue

        fieldName = data.colID[colIdx]
        fieldRule = ProtoHelper.getProtoRule(data.colType[colIdx])
        fieldType = ProtoHelper.getProtoType(data.colType[colIdx])

        # 根据 fieldRule 和 fieldType 设置字段值
        # (简化示意, 实际需要处理 Vector/KVP 等复合类型)
        if fieldRule == "repeated":
            if isinstance(cellVal, list):
                getattr(item, fieldName).extend(cellVal)
            elif isinstance(cellVal, dict):
                # map 类型: 转为 repeated KVP
                KVPClass = getattr(pb2_module, fieldType)
                for k, v in cellVal.items():
                    kvp = KVPClass()
                    kvp.k = k
                    kvp.v = v
                    getattr(item, fieldName).append(kvp)
            else:
                if fieldType in ("Vector2Int", "Vector2Float"):
                    field = getattr(item, fieldName)
                    field.x = cellVal[0]
                    field.y = cellVal[1]
                elif fieldType in ("Vector3Int", "Vector3Float"):
                    field = getattr(item, fieldName)
                    field.x = cellVal[0]
                    field.y = cellVal[1]
                    field.z = cellVal[2]
                else:
                    setattr(item, fieldName, cellVal)

```

```

entry_bytes = item.SerializeToString()
entry_len = len(entry_bytes)
if entry_len > 0xFFFF:
    raise Exception("单条数据超过 65535 字节限制")

offsets.append(entries_buffer.tell())
entries_buffer.write(struct.pack('>H', entry_len))
entries_buffer.write(entry_bytes)

# ---- Stage 2: 生成 Header ----
# (根据主键类型选择对应的 Header 消息类型)
mainKeyType = ProtoHelper.getProtoType(data.colType[mainKeyCol])
headerClassName = "Config" + mainKeyType.replace("32", "").capitalize() + "Header"
HeaderClass = getattr(pb2_module, headerClassName, None)

# 如果 pb2 中没有对应的 Header 类, 使用 LIST 模式 (旧版兼容)
if HeaderClass is None:
    # 旧版: 直接序列化整个列表
    listObj = ListClass()
    for rowIdx in range(rowCount):
        row = data.data[rowIdx]
        item = listObj.items.add()
        # ... 填充字段 (同上)
    with open(path, 'wb') as f:
        f.write(listObj.SerializeToString())
    return True

config_header = HeaderClass()
mainKeyCount = 0
mainKeyDict = {}

for rowIdx in range(rowCount):
    row = data.data[rowIdx]
    entry = config_header.headers.add()
    entry.id = row[mainKeyCol]
    if subKeyCol is not None:
        entry.subId = row[subKeyCol]
    if entry.id not in mainKeyDict:
        mainKeyDict[entry.id] = True
        mainKeyCount += 1

```

```

else:
    mainKeyCount += 1
    entry.offset = offsets[rowIdx]

# ---- Stage 3: 写入文件 ----
header_bytes = config_header.SerializeToString()
header_length = len(header_bytes)
header_count = len(config_header.headers)

with open(path, 'wb') as f:
    f.write(struct.pack('>H', mainKeyCount))
    f.write(struct.pack('>H', header_count))
    f.write(struct.pack('>I', header_length))
    f.write(header_bytes)
    f.write(entries_buffer.getvalue())

return True
----- 代码 -----

```

◆ 5.6 CSWriter.py — 生成 C# 代码

```

----- 代码(python) -----
# gtp/writer/CSWriter.py

from gtp.writer import ProtoHelper

NAMESPACE = "com.youzu.got.config"
MANAGER_CLASS = "GotConfigManager"

# ===== 单键模板 =====
TEMPLATE_SINGLE_KEY = '''/**
 * Generated By Tool, DO NOT MODIFY.
 **/

using System;
using System.Collections.Generic;
using System.IO;
using UnityEngine;

```

```

namespace {namespace}
{
    public partial class {configClass}
    {
        public bool isLoading;
        private Dictionary<{keyType}, {protoClass}> m_items = null;

        public Dictionary<{keyType}, {protoClass}> items
        {
            get
            {
                Load();
                return m_items;
            }
        }

        public void Load()
        {
            if (isLoading) return;
            m_items = new Dictionary<{keyType}, {protoClass}>();
            isLoading = true;

            // 从 .bytes 加载二进制数据并反序列化
            var bytes = LoadConfigBytes("{bytesName}");
            if (bytes != null)
            {
                using (var ms = new MemoryStream(bytes))
                {
                    var list = Deserialize<{protoClass}_LIST_TOOL_RESERVED>(ms);
                    foreach (var item in list.items)
                    {
                        m_items[item.{keyField}] = item;
                    }
                }
            }
        }

        public {protoClass} Get({keyType} key)
        {

```

```

        Load();
        {protoClass} item = null;
        items.TryGetValue(key, out item);
        return item;
    }
}

```

```

public partial class {managerClass}
{
    private {configClass} m_{configVar};
    public {configClass} {configVar}
    {
        get
        {
            if (m_{configVar} == null) m_{configVar} = new {configClass}();
            return m_{configVar};
        }
    }
}
}

```

===== 双键子类模板 (Tree) =====

```

TEMPLATE_DOUBLE_KEY_CLASS = "public class C{protoClass}
{
    public Dictionary<{keyType2}, {protoClass}> items = new Dictionary<{keyType2}, {protoClass}>();
    public {protoClass} main;

    public {protoClass} Get({keyType2} key)
    {
        {protoClass} item = null;
        items.TryGetValue(key, out item);
        return item;
    }
}"

```

===== 双键模板 (Tree) =====

```

TEMPLATE_DOUBLE_KEY_TREE = "/*
* Generated By Tool, DO NOT MODIFY.

```

```
**/
```

```
using System;  
using System.Collections.Generic;  
using System.IO;  
using UnityEngine;
```

```
namespace {namespace}
```

```
{{
```

```
{subclass}
```

```
public partial class {configClass}
```

```
{{
```

```
    public bool isLoading;
```

```
    private Dictionary<{keyType1}, Dictionary<{keyType2}, {protoClass}>> m_items = null;
```

```
    private Dictionary<{keyType1}, C{protoClass}> m_citems = null;
```

```
    public Dictionary<{keyType1}, Dictionary<{keyType2}, {protoClass}>> items
```

```
    {{
```

```
        get {{ Load(); return m_items; }}
```

```
    }}
```

```
    public Dictionary<{keyType1}, C{protoClass}> citems
```

```
    {{
```

```
        get {{ Load(); return m_citems; }}
```

```
    }}
```

```
    public void Load()
```

```
    {{
```

```
        if (isLoading) return;
```

```
        m_items = new Dictionary<{keyType1}, Dictionary<{keyType2}, {protoClass}>>();
```

```
        m_citems = new Dictionary<{keyType1}, C{protoClass}>();
```

```
        isLoading = true;
```

```
        var bytes = LoadConfigBytes("{bytesName}");
```

```
        if (bytes != null)
```

```
        {{
```

```
            using (var ms = new MemoryStream(bytes))
```

```
            {{
```

```
                var list = Deserialize<{protoClass}_LIST_TOOL_RESERVED>(ms);
```

```

foreach (var item in list.items)
{
    Dictionary<{keyType2}, {protoClass}> sd = null;
    if (!m_items.TryGetValue(item.{keyField1}, out sd))
    {
        sd = new Dictionary<{keyType2}, {protoClass}>();
        m_items[item.{keyField1}] = sd;
    }
    sd[item.{keyField2}] = item;

    if (!m_citems.ContainsKey(item.{keyField1}))
    {
        var ci = new C{protoClass}();
        ci.items = sd;
        ci.main = item;
        m_citems[item.{keyField1}] = ci;
    }
}
}
}
}
}

```

```

public Dictionary<{keyType2}, {protoClass}> Get({keyType1} key)
{
    Load();
    Dictionary<{keyType2}, {protoClass}> item = null;
    items.TryGetValue(key, out item);
    return item;
}

```

```

public C{protoClass} GetId({keyType1} key)
{
    Load();
    C{protoClass} item = null;
    citems.TryGetValue(key, out item);
    return item;
}

```

```

public {protoClass} Get({keyType1} key1, {keyType2} key2)
{

```

```

        Load();
        var d = Get(key1);
        if (d != null)
        {
            {protoClass} item = null;
            d.TryGetValue(key2, out item);
            return item;
        }
        return null;
    }
}

```

```

public partial class {managerClass}
{
    private {configClass} m_{configVar};
    public {configClass} {configVar}
    {
        get
        {
            if (m_{configVar} == null) m_{configVar} = new {configClass}();
            return m_{configVar};
        }
    }
}
}
'''

```

===== 双键模板 (Row, 第二键为自增 int) =====

```

TEMPLATE_DOUBLE_KEY_ROW_CLASS = '''public class C{protoClass}
{
    public Dictionary<int, {protoClass}> items = new Dictionary<int, {protoClass}>();
    public {protoClass} main;

    public {protoClass} Get(int key)
    {
        {protoClass} item = null;
        items.TryGetValue(key, out item);
        return item;
    }
}'''

```



```

TEMPLATE_DOUBLE_KEY_ROW = ""/**
 * Generated By Tool, DO NOT MODIFY.
 **/

using System;
using System.Collections.Generic;
using System.IO;
using UnityEngine;

namespace {namespace}
{{
    {subclass}

    public partial class {configClass}
    {{
        public bool isLoading;
        private Dictionary<{keyType1}, Dictionary<int, {protoClass}>> m_items = null;
        private Dictionary<{keyType1}, C{protoClass}> m_citems = null;

        public Dictionary<{keyType1}, Dictionary<int, {protoClass}>> items
        {{
            get {{ Load(); return m_items; }}
        }}

        public void Load()
        {{
            if (isLoading) return;
            m_items = new Dictionary<{keyType1}, Dictionary<int, {protoClass}>>();
            m_citems = new Dictionary<{keyType1}, C{protoClass}>();
            isLoading = true;

            var bytes = LoadConfigBytes("{bytesName}");
            if (bytes != null)
            {{
                using (var ms = new MemoryStream(bytes))
                {{
                    var list = Deserialize<{protoClass}_LIST_TOOL_RESERVED>(ms);
                    foreach (var item in list.items)
                    {{

```

```

Dictionary<int, {protoClass}> sd = null;
if (!m_items.TryGetValue(item.{keyField1}, out sd))
{
    sd = new Dictionary<int, {protoClass}>();
    m_items[item.{keyField1}] = sd;
}
sd[sd.Count + 1] = item;

if (!m_citems.ContainsKey(item.{keyField1}))
{
    var ci = new C{protoClass}();
    ci.items = sd;
    ci.main = item;
    m_citems[item.{keyField1}] = ci;
}
}
}
}
}
}

```

```

public Dictionary<int, {protoClass}> Get({keyType1} key)
{
    Load();
    Dictionary<int, {protoClass}> item = null;
    items.TryGetValue(key, out item);
    return item;
}

```

```

public {protoClass} Get({keyType1} key1, int key2)
{
    Load();
    var d = Get(key1);
    if (d != null)
    {
        {protoClass} item = null;
        d.TryGetValue(key2, out item);
        return item;
    }
    return null;
}

```

```

}}

public partial class {managerClass}
{{
    private {configClass} m_{configVar};
    public {configClass} {configVar}
    {{
        get
        {{
            if (m_{configVar} == null) m_{configVar} = new {configClass}();
            return m_{configVar};
        }}
    }}
}}
}}
'''

```

```

def writeData(path, data):
    """根据索引键数量选择模板，生成 C# 配置类"""
    colCount = len(data.colID)
    if colCount <= 0 or len(data.dataIndex) == 0:
        return False

    protoClass = data.id[0].upper() + data.id[1:]

    fmt = {
        "namespace":  NAMESPACE,
        "configClass": protoClass + "Config",
        "configVar":   data.id[0].lower() + data.id[1:] + "Config",
        "protoClass":  protoClass,
        "managerClass": MANAGER_CLASS,
        "bytesName":   protoClass,
    }

    keyCount = len(data.dataIndex[0])

    if keyCount <= 1:
        # ---- 单键 ----
        keyColType = data.colType[data.colID.index(data.dataIndex[0][0])]

```

```

fmt["keyType"] = ProtoHelper.getProtoType(keyColType).replace("32", "")
fmt["keyField"] = data.dataIndex[0][0]
content = TEMPLATE_SINGLE_KEY.format(**fmt)

```

else:

```

# ---- 双键 ----
keyColType1 = data.colType[data.colID.index(data.dataIndex[0][0])]
keyColType2 = data.colType[data.colID.index(data.dataIndex[0][1])]
fmt["keyType1"] = ProtoHelper.getProtoType(keyColType1).replace("32", "")
fmt["keyType2"] = ProtoHelper.getProtoType(keyColType2).replace("32", "")
fmt["keyField1"] = data.dataIndex[0][0]
fmt["keyField2"] = data.dataIndex[0][1]

```

if "R" in data.ext:

```

# Row 模式: 第二键为自增 int
fmt["subclass"] = TEMPLATE_DOUBLE_KEY_ROW_CLASS.format(**fmt)
content = TEMPLATE_DOUBLE_KEY_ROW.format(**fmt)

```

else:

```

# Tree 模式
fmt["subclass"] = TEMPLATE_DOUBLE_KEY_CLASS.format(**fmt)
content = TEMPLATE_DOUBLE_KEY_TREE.format(**fmt)

```

with open(path, 'w', encoding='utf-8') as f:

```

f.write(content)

```

return True

----- 代码 -----

◆ 5.7 publish_config.py — 入口脚本

----- 代码(python) -----

```

# publish_config.py

```

```

import os
import sys
import configparser
from gtp.common.ConfigData import splitConfigClient
from gtp.reader.ExcelReader import readExcel
from gtp.writer import ProtoWriter, CSWriter

```

```

def loadConfig():
    """从 tools_config.ini 加载路径配置"""
    config = configparser.ConfigParser()
    config.read("tools_config.ini", encoding="utf-8")
    return {
        "config_dir": config.get("path", "config_dir"),
        "client_dir": config.get("path", "client_dir"),
        "proto_dir": config.get("path", "proto_dir"),
    }

def publishFile(filePath, cfg):
    """发布单个 Excel 配置文件"""
    fileID = os.path.splitext(os.path.basename(filePath))[0]
    print("Publishing: " + fileID)

    # 1. 读取 Excel
    rawData = readExcel(filePath)
    if rawData is None:
        print(" Skip (no data): " + fileID)
        return False

    # 2. 仅保留客户端列
    clientData = splitConfigClient(rawData)
    if len(clientData.colID) == 0 or len(clientData.dataIndex) == 0:
        print(" Skip (no client columns): " + fileID)
        return False

    className = fileID[0].upper() + fileID[1:]

    # 3. 生成 .proto
    protoPath = os.path.join(cfg["proto_dir"], fileID + ".proto")
    ProtoWriter.writeData(protoPath, clientData)
    print(" -> " + protoPath)

    # 4. 生成 .cs
    csPath = os.path.join(cfg["client_dir"], "ScriptGenerated", "Config", className + "Config.cs")
    os.makedirs(os.path.dirname(csPath), exist_ok=True)

```

```
CSWriter.writeData(csPath, clientData)
```

```
print(" -> " + csPath)
```

```
# 5. 编译 proto 并生成 .bytes (需要 protoc + pb2 模块)
```

```
# 此处需要先运行 protoc 编译 .proto → _pb2.py
```

```
# 然后 import pb2 模块并调用 ProtoDataWriter.writeData()
```

```
# 具体流程见第七节
```

```
return True
```

```
def publishAll(cfg):
```

```
    """遍历目录发布所有 Excel 文件"""
```

```
    configDir = os.path.join(cfg["config_dir"], "3xlsx")
```

```
    count = 0
```

```
    for root, dirs, files in os.walk(configDir):
```

```
        # 跳过 _temp 目录
```

```
        dirs[:] = [d for d in dirs if d != "_temp"]
```

```
        for f in files:
```

```
            ext = os.path.splitext(f)[1].lower()
```

```
            if ext not in (".xls", ".xlsx", ".xlsm"):
```

```
                continue
```

```
            if "~" in f or "$" in f:
```

```
                continue # 跳过临时文件
```

```
            filePath = os.path.join(root, f)
```

```
            if publishFile(filePath, cfg):
```

```
                count += 1
```

```
    print("\nDone. Published %d files." % count)
```

```
if __name__ == "__main__":
```

```
    cfg = loadConfig()
```

```
# 确保输出目录存在
```

```
os.makedirs(cfg["proto_dir"], exist_ok=True)
```

```

if len(sys.argv) > 1:
    # 发布指定文件
    for path in sys.argv[1:]:
        publishFile(path, cfg)
else:
    # 发布全部
    publishAll(cfg)
----- 代码 -----

```

◆ 5.8 ConfigManager.py — 配置管理（精简版）

```

----- 代码(python) -----
# gtp/common/ConfigManager.py

import configparser
import os

class ConfigManager:
    """从.ini 文件读取路径配置"""

    def __init__(self, iniPath="tools_config.ini"):
        self.config = configparser.ConfigParser()
        self.config.read(iniPath, encoding="utf-8")

    @property
    def config_dir(self):
        return self.config.get("path", "config_dir")

    @property
    def client_dir(self):
        return self.config.get("path", "client_dir")

    @property
    def proto_dir(self):
        return self.config.get("path", "proto_dir")
----- 代码 -----

```

■ 六、配置文件

◆ tools_config.ini

----- 代码(ini) -----

[path]

Excel 配置表根目录（内含 3xls/ 子目录）

config_dir = F:\GotPCClient\excel

Unity 客户端工程目录

client_dir = F:\GotPCClient\clientproject\GOT_PC\Assets

Proto 文件输出目录

proto_dir = F:\GotPCClient\tools_publish\proto

protoc 编译器路径

protoc = F:\GotPCClient\tools\3rdparty\protobuf\protoc.exe

[publish]

跳过的文件名（逗号分隔）

skip_files = _配置说明.xlsx

----- 代码 -----

■ 七、Bat 启动项

◆ publish_config.bat — 发布全部

----- 代码(bat) -----

@echo off

chcp 65001 >nul

echo =====


```

echo 配表工具 - 发布全部
echo =====

REM 发布全部 Excel 配置表
python publish_config.py

REM 编译所有 .proto 文件为 C# 代码（用于运行时反序列化）
echo.
echo Compiling proto files...
for %%f in (_publish\proto\*.proto) do (
    3rdparty\protobuf\protoc.exe --proto_path=_publish\proto --csharp_out=_publish\cs_proto
    "%%f"
)

echo.
echo Done!
pause
----- 代码 -----

```

◆ publish_single.bat — 发布单个表

```

----- 代码(bat) -----
@echo off
chcp 65001 >nul

if "%~1"==" " (
    echo Usage: publish_single.bat [excel_file_path]
    pause
    exit /b 1
)

python publish_config.py %*
pause
----- 代码 -----

```

■ 八、Protoc 编译流程

发布 .bytes 数据需要先将 .proto 编译为 Python pb2 模块:

----- 代码(bat) -----

REM 1. 先编译 ConfigBaseType.proto

protoc --proto_path=_publish/proto --python_out=_publish/pb2 ConfigBaseType.proto

REM 2. 编译各表的 .proto

protoc --proto_path=_publish/proto --python_out=_publish/pb2 hero.proto

REM 3. 然后在 Python 中 import 使用

REM import _publish.pb2.hero_pb2 as hero_pb2

REM ProtoDataWriter.writeData("hero.bytes", clientData, hero_pb2)

----- 代码 -----

完整流程建议在 publish_config.py 中自动化:

----- 代码(python) -----

import subprocess

def compileProto(protoDir, outputDir, protocPath):

"""编译所有 .proto 文件为 Python pb2 模块"""

os.makedirs(outputDir, exist_ok=True)

for f in os.listdir(protoDir):

if f.endswith(".proto"):

subprocess.run([

protocPath,

"--proto_path=" + protoDir,

"--python_out=" + outputDir,

f

], check=True)

----- 代码 -----

■ 九、C# 端读取示例

生成的 C# 类在客户端中的使用方式：

```
----- 代码(csharp) -----  
  
// 单键表  
var heroConfig = GotConfigManager.heroConfig;  
heroConfig.Load();  
var hero = heroConfig.Get(1001); // 通过 ID 获取  
Debug.Log(hero.hero_name);  
  
// 双键树形表  
var skillConfig = GotConfigManager.skillConfig;  
var skills = skillConfig.Get(1001);    // 获取 key1 下所有子项  
var skill = skillConfig.Get(1001, 2);  // 获取指定 key1+key2  
var group = skillConfig.GetId(1001);   // 获取包装对象 (含 .main)  
  
// 遍历所有  
foreach (var kv in heroConfig.items)  
{  
    Debug.Log(kv.Key + " -> " + kv.Value.hero_name);  
}  
----- 代码 -----
```

■ 十、快速上手步骤

1. 安装依赖

```
----- 代码(bash) -----  
  
pip install openpyxl protobuf  
----- 代码 -----
```

2. 下载 protoc.exe 放到 3rdparty/protobuf/

3. 创建目录结构（按第三节）

4. 编写 tools_config.ini（按第六节，修改为你的路径）

5. 放入 Excel 配置表 到 config_dir/3xlsx/ 目录

6. 运行

----- 代码(bash) -----

```
python publish_config.py
```

----- 代码 -----

或双击 publish_config.bat

7. 产出

- _publish/proto/*.proto — Protobuf 定义
- Assets/ScriptGenerated/Config/*Config.cs — C# 配置类
- Assets/ResourcesAssets/Config/Data/Default/*.bytes — 二进制数据

■ 附录：类型速查表

| Excel 类型 | Proto 类型 | Proto 规则 | C# Key 类型 |

|-----|-----|-----|-----|

| int | int32 | optional | int |

| uint | uint32 | optional | uint |

| float | float | optional | - |

| bool | int32 | optional | - |

| string | string | optional | string |

| vec2.int | Vector2Int | optional | - |

| vec2.float | Vector2Float | optional | - |

| vec3.int | Vector3Int | optional | - |

| vec3.float | Vector3Float | optional | - |

| array.int | int32 | repeated | - |

| array.uint | uint32 | repeated | - |

| array.float | float | repeated | - |

| array.string | string | repeated | - |

| array.bool | int32 | repeated | - |

| array.vec2.int | Vector2Int | repeated | - |

| array.vec2.float | Vector2Float | repeated | - |

| array.vec3.int | Vector3Int | repeated | - |

| array.vec3.float | Vector3Float | repeated | - |

| map.string.int | KeyValuePairStrInt | repeated | - |

| map.string.float | KeyValuePairStrFloat | repeated | - |